

Trabajo Práctico N° 2

Estructuras de Datos y Algoritmos II

Secuencias

Integrantes:

Becerra, Nicolás

Deppen, Nahuel

Messana Gulielmi, Agustín

Junio de 2026

Consigna

b) Para la implementación dada en a, dar la especificación (trabajo y profundidad) de las funciones filterS, reduceS y scanS. No es necesario demostrarlo.

Especificación de costos de filterS

Implementación

```
filterL :: (a -> Bool) -> [a] -> [a]
filterL p [] = []
filterL p (x : xs) =
    let (px, xs') = p x ||| filterL p xs
    in if px then x : xs' else xs'
```

Trabajo de filterS (W)

Para una lista, el trabajo se define como:

$$W(\text{filterL } p []) = k_1$$

$$W(\text{filterL } p (x : xs)) = W(p \ x) + W(\text{filterL } p \ xs) + k_2$$

Donde k_1 y k_2 son constantes que representan operaciones básicas. Luego, para una secuencia s :

$$W(\text{filterL } p \ s) = \sum_{i=0}^{|s|-1} W(p \ s_i) + |s| * k_2$$

Por lo tanto:

$$W(\text{filterL } p \ s) = O(|s| + \sum_{i=0}^{|s|-1} W(p \ s_i))$$

Profundidad de filterS (S)

La profundidad la calculamos evaluando en paralelo el predicado y el llamado recursivo

$$S(\text{filterL } p []) = k_1$$

$$S(\text{filterL } p (x : xs)) = \max(S(p \ x), S(\text{filterL } p \ xs)) + k_2$$

Donde k_2 representa el tiempo constante de concatenar los resultados en la estructura secuencial de la lista.

Por lo tanto:

$$S(\text{filterL } p \ s) = O\left(|s| + \max_{0 \leq i \leq |s|-1} S(p \ s_i)\right)$$

Especificación de costos de contrL

Implementación

```
contrL :: (a -> a -> a) -> [a] -> [a]
contrL f [] = []
```

```

contrL f [x] = [x]
contrL f (x1 : x2 : xs) =
  let (y, ys) = (f x1 x2) ||| (contrL f xs)
  in y : ys

```

Trabajo de contrL (W)

Tenemos que:

$$W(\text{contrL } f []) = k_1$$

$$W(\text{contrL } f [x]) = k_2$$

$$W(\text{contrL } f (x_1 : x_2 : xs)) = W(f x_1 x_2) + W(\text{contrL } f xs) + k_3$$

Donde k_1 , k_2 y k_3 son constantes. Luego:

$$W(\text{contrL } f (x_1 : x_2 : xs)) = \sum_{i=0}^{\left\lfloor \frac{|xs|}{2} \right\rfloor} W(f xs_{2i} xs_{2i+1}) + \left\lfloor \frac{|xs|}{2} \right\rfloor * k_3$$

Por lo tanto:

$$W(\text{contrL } f (x_1 : x_2 : xs)) = O\left(\sum_{i=0}^{\left\lfloor \frac{|xs|}{2} \right\rfloor} W(f xs_{2i} xs_{2i+1})\right)$$

Profundidad de contrL (S)

La profundidad la calculamos como:

$$S(\text{contrL } f []) = k_1$$

$$S(\text{contrL } f [x]) = k_2$$

$$S(\text{contrL } f (x_1 : x_2 : xs)) = \max(S(f x_1 x_2), S(\text{contrL } f xs)) + k_3$$

Luego,

$$S(\text{contrL } f (x_1 : x_2 : xs)) = \max_{0 \leq i \leq \left\lfloor \frac{|xs|}{2} \right\rfloor} S(f xs_{2i} xs_{2i+1}) + \left\lfloor \frac{|xs|}{2} \right\rfloor * k_3$$

Por lo tanto:

$$S(\text{contrL } f (x_1 : x_2 : xs)) = O\left(\max_{0 \leq i \leq \left\lfloor \frac{|xs|}{2} \right\rfloor} S(f xs_{2i} xs_{2i+1}) + |xs|\right)$$

Especificación de costos de reduceL

Implementación

```

reduceL :: (a -> a -> a) -> a -> [a] -> a
reduceL f e [] = e
reduceL f e [x] = f e x
reduceL f e xs = reduceL f e (contrL f xs)

```

Trabajo de reduceL (W)

$$W(\text{reduceL } f e []) = k_1$$

$$W(\text{reduceL f e [x]}) = W(\text{f e x}) + k_2$$

$$W(\text{reduceL f e xs}) = W(\text{reduceL f e (contrL f xs)}) + W(\text{contrL f xs}) + k_3$$

Donde k_1 , k_2 y k_3 son constantes. Por lo tanto:

$$W(\text{reduceL f e xs}) = O\left(\sum_{(f \ x \ y) \in \text{Or}(f, e, xs)} W(f \ x \ y)\right)$$

Profundidad de reduceL (S)

$$S(\text{reduceL f e []}) = k_1$$

$$S(\text{reduceL f e [x]}) = S(\text{f e x}) + k_2$$

$$S(\text{reduceL f e xs}) = S(\text{contrL f xs}) + S(\text{reduceL f e (contrL f xs)}) + k_3$$

Por lo tanto:

$$S(\text{reduceL f e xs}) = O\left(\max_{(f \ x \ y) \in \text{Or}(f, e, xs)} S(f \ x \ y) * \log_2(|xs|) + |xs|\right)$$

Especificación de costos de scanL

Implementación

```
scanL :: (a -> a -> a) -> a -> [a] -> ([a], a)
```

```
scanL f b [] = ([], b)
```

```
scanL f b [x] = ([b], f b x)
```

```
scanL f b s =
```

```
  let (s', r) = scanL f b (contrL f s)
```

```
  in (expandL f b s s', r)
```

Trabajo de scanL (W)

Definimos el trabajo como:

$$W(\text{scanL f b []}) = k_1$$

$$W(\text{scanL f b [x]}) = W(\text{f b x}) + k_2$$

$$W(\text{scanL f b s}) = W(\text{contrL f s}) + W(\text{scanL f b (contrL f s)}) + W(\text{expandL f b s s'}) + k_3$$

Donde:

- $W(\text{contrL f s}) = O\left(\sum_{i=0}^{\left\lceil \frac{n}{2} \right\rceil - 1} W(f \ s_{2i} \ s_{2i+1})\right)$
- $W(\text{expandL f b s s'}) = O(n)$ (asumiendo que procesa cada elemento una vez)

La recurrencia para el trabajo es:

$$W(n) = W\left(\frac{n}{2}\right) + O(n)$$

Cuyo resultado es:

$$W(\text{scanL } f \text{ b } s) = O\left(\sum_{(f \times y) \in Os(f, b, s)} W(f \times y) + n\right)$$

Profundidad de scanL (S)

Definimos la profundidad como:

$$S(\text{scanL } f \text{ b } []) = k_1$$

$$S(\text{scanL } f \text{ b } [x]) = S(f \text{ b } x) + k_2$$

$$S(\text{scanL } f \text{ b } s) = S(\text{contrL } f \text{ s}) + S(\text{scanL } f \text{ b } (\text{contrL } f \text{ s})) + S(\text{expandL } f \text{ b } s \text{ s}') + k_3$$

Donde:

- $S(\text{contrL } f \text{ s}) = O\left(\max_{0 \leq i \leq \left\lfloor \frac{n}{2} \right\rfloor - 1} S(f \text{ s}_{2i} \text{ s}_{2i+1}) + \log_2 n\right)$
- $S(\text{expandL } f \text{ b } s \text{ s}') = O(\log_2 n)$ (asumiendo paralelismo en la expansión)

La recurrencia para la profundidad es:

$$S(n) = S\left(\frac{n}{2}\right) + O(\log_2 n)$$

Cuyo resultado es:

$$S(\text{scanL } f \text{ b } s) = O\left(\max_{(f \times y) \in Os(f, b, s)} S(f \times y) * \log_2 n + (\log_2 n)^2\right)$$

Consigna

d) Muestre que la implementación dada en c verifica la especificación de costos para la profundidad de operación reduceS.

Veamos la profundidad de reduceA.

- La profundidad de nthA es constante.
- La profundidad de tabulateA es $O\left(\max_{0 \leq i \leq n-1} S(f \text{ i})\right)$

En nuestra implementación de contrA

```
contrA :: (a -> a -> a) -> A.Array a -> A.Array a
```

```
contrA f arr = tabulateA faux m
```

```
where
```

```
  faux i = if i == (mid) then nthA arr (n - 1) else faux' i
```

```
  faux' i = f (nthA arr (i * 2)) (nthA arr (i * 2 + 1))
```

```
  mid = n `div` 2
```

```
  n = lengthA arr
```

```
  m = if n `mod` 2 == 0 then mid else mid + 1
```

Se usa `tabulateA` con argumento `faux` y tamaño $m = \left\lceil \frac{n}{2} \right\rceil$; la profundidad de `faux` depende de la función `f` (nuestro operador $\oplus$). Como por consigna debemos asumir que $\oplus \in O(1)$, entonces la profundidad de `faux` es $O(1)$. Por lo tanto, el costo de `contraer` es constante: $S(\text{contraer } f \text{ arr}) = O(1)$.

La función `reduceA` se implementa como:

```
reduceA :: (a -> a -> a) -> a -> A.Arr a -> a
reduceA f e arr = case (lengthA arr) of
  0 -> e
  1 -> f e (nthA arr 0)
  n -> reduceA f e (contraer f arr)
```

El llamado recursivo procesa un arreglo que fue reducido a la mitad de su tamaño original. La ecuación de recurrencia para la profundidad total queda determinada por el costo de `contraer` más la llamada recursiva:

$$S(n) = S\left(\frac{n}{2}\right) + O(1)$$

Para resolver la recurrencia, aplicamos el Teorema Maestro de la siguiente forma:

$$T(n) = a * T\left(\frac{n}{b}\right) + f(n)$$

Donde:

$$a = 1$$

$$b = 2$$

$$f(n) \in O(1)$$

$$\text{Calculamos } \log_b a = \log_2 1 = 0.$$

Como $n^{\log_b a} = n^0 = 1$, estamos en el caso del Teorema donde $f(n) \in \Theta(n^{\log_b a})$

Por lo tanto, la solución final es:

$$S(n) \in O(\log_2 n)$$

De esta manera, queda demostrado que la implementación verifica la cota logarítmica exigida.
